

NLP

BAF agents are powered by NLP (Natural Language Processing) tools that provide them with the necessary capabilities to understand and generate human language.

- **Intent Classification** to recognize the user intentions, implicit in messages.
- **Named Entity Recognition (NER)** to identify entities within the user messages.
- **Large Language Models (LLMs)** that can be used for many NLP tasks.
- **Retrieval Augmented Generation (RAG)** to query your database with natural language.
- **Speech-to-text** to transcribe audio messages, allowing the users to use their voice to interact with the agents.

Intent classification

When a user sends a message to the agent, it needs to recognize its intention among a set of [intents](#). In NLP, this problem is known as Text Classification.

Given an user message, the agent must classify it into one of the possible intents.

To successfully do this, it is necessary to provide a set of example sentences for each intent (this is the training data).

In this section, we delve into the different intent classifiers available in BAF. Each one has its pros and cons.

Each agent state has its own intent classifier. This is because each intent classifier is trained to recognize only those intents allowed in its state. This way, you can choose the intent classifier that better suits for each state depending on the needs. Let's see how to configure this.

Configuration

Each state has an [intent classifier configuration](#), that defines its intent classifier's properties.

This is how you can create an intent classifier configuration:

```
from baf.nlp.intent_classifier.intent_classifier_configuration import
SimpleIntentClassifierConfiguration

agent = Agent('example_agent')

simple_config = SimpleIntentClassifierConfiguration(
    framework='pytorch',
    activation_last_layer='sigmoid',
    lr=0.001
)
```

We can set a configuration for a specific state:

```
example_state = agent.new_state('example_state', ic_config=simple_config)
```

Or a default configuration for all states (note that this would replace any previously defined configuration):

```
agent.set_default_ic_config(simple_config)
```

Note

If you don't specify any configuration for a state, the [Simple Intent Classifier](#) (PyTorch implementation) will be selected by default.

Note

All intent classifier configuration classes implement the [IntentClassifierConfiguration](#) class and all intent classifiers implement the [IntentClassifier](#) class

Simple Intent Classifier

The Simple Intent Classifier is based on a basic neural network as the prediction model. It is trained with the intent's training sentences. When running, it is able to predict the intent of a message if it is close to any of the training sentences.

We provide 2 implementations of the same intent classifier:

- [SimpleIntentClassifierTorch](#): PyTorch-based implementation. It uses a [PyTorch module](#) as the prediction model.
- [SimpleIntentClassifierTF](#): Tensorflow-based implementation. It uses a [Keras](#) neural network as the prediction model.

Note

Even though both intent classifiers implement the same neural network architecture, the performance may not be equal.

Also, the training process is not deterministic and performance may vary between different agent executions.

The neural network architecture can be customized.

The Simple Intent Classifier is based on a basic neural network as the prediction model. It is trained with the intent's training sentences. When running, it is able to predict the intent of a message if it is close to any of the training sentences.

You can see all the configuration possibilities of this intent classifier here (including the framework selection): [SimpleIntentClassifierConfiguration](#)

The [NLP_PRE_PROCESSING](#) agent property influences the performance of this intent classifier. If you decide to preprocess the user messages (this is done before the intent prediction), the intent predictions will probably be more accurate.

When to use it?

- If you want a very light, customizable and quickly trainable intent classifier.

- If you are certain of how the user messages will look like.
- If you want to restrict the user's writing freedom, forcing him/her to write in a particular way or to choose from a predefined set of messages (with buttons).

Pros

- Free
- Fast training
- Fast predictions
- Very small

Cons

- You need to provide training sentences (quantity and quality increases the success probabilities)
- Not possible to understand semantic similarities, only word similarities. For example, if a training sentence is 'yes' and the user says 'of course' (something not present in the training sentences), the prediction will fail.
- If 2 or more intents have very similar training sentences, probabilities of wrong predictions increase

Example scenario

Imagine your agent has a state where it asks some question to the user, expecting a yes/no answer:

```
yes_intent = agent.new_intent('yes_intent', ['Yes'])

no_intent = agent.new_intent('no_intent', ['No'])

example_state = agent.new_state('example_state',
ic_config=SimpleIntentClassifierConfiguration())

def example_body(session: Session):
    websocket_platform.reply(session, 'Do you want to continue talking?')
    websocket_platform.reply_options(session, ['Yes', 'No'])

example_state.set_body(example_body)
example_state.when_intent_matched(yes_intent).go_to(state1)
example_state.when_intent_matched(no_intent).go_to(state2)
```

In this kind of situations, the Simple Intent Classifier will satisfy the agent needs. You can also remove the reply_options message and let the user write, although if you want to force the user reply, this is strongly recommended.

LLM Intent Classifier

The [LLMIntentClassifier](#) uses a [Large Language Model \(LLM\)](#) to predict the intent of a message. LLMs are multimodal models that can solve a wide variety of tasks just by providing them the right prompts in natural language. In this case, we can ask them to classify a sentence into the appropriate intent.

You can see all the configuration possibilities of this intent classifier here:

[LLMIntentClassifierConfiguration](#)

When to use it?

- If you want a powerful and very smart intent classifier.
- If you don't quite know how the user messages will look like.
- If you want to give the users writing freedom.

Pros

- No need to train it. It is a general purpose model.
- Through API, no need to host it (also possible to run it locally with HuggingFace)
- No need for training sentences, just intent descriptions (you can also use both).
- Understands semantic similarities. For example, if a training sentence is 'yes' and the user says 'of course' (something not present in the training sentences), the prediction will probably hit.
- You can select any LLM you want (even different LLMs for each state)
- Powerful Named Entity Recognition integrated.

Cons

- APIs not free to use
- LLMs are big (this can only affect you if you run them locally)
- Some predictions may be slow (a few seconds)

Example scenario

Imagine your agent has a state where some of the possible intents is a 'help' intent, intended to guide the user on how to use the agent. Since there are a lot of different ways the user could ask for help, and we don't want to think about them all, we can simply provide an intent description and use the LLM Intent Classifier:

```
from baf.nlp.intent_classifier.intent_classifier_configuration import
LLMIntentClassifierConfiguration
from baf.nlp.llm.llm_openai_api import LLMOpenAI

agent = Agent('example_agent')
llm = LLMOpenAI(agent=agent, name='gpt-4o-mini')

ic_config = LLMIntentClassifierConfiguration(
    llm_name='gpt-4o-mini',
    parameters={
        "seed": None,
        "top_p": 1,
        "temperature": 1,
    },
    use_intent_descriptions=True,
    use_training_sentences=False,
    use_entity_descriptions=True,
```

```

        use_entity_synonyms=False
    )

    help_intent = agent.new_intent(
        name='help_intent',
        description='The user needs help to be able to use the agent properly or to
        find some information'
    )

    example_state = agent.new_state('example_state', ic_config=ic_config)

    def example_body(session: Session):
        # ...

    example_state.set_body(example_body)
    example_state.when_intent_matched(intent1).go_to(state1)
    # ...
    example_state.when_intent_matched(help_intent).go_to(help_state)

```

Named Entity Recognition

NER is a Natural Language Processing task that involves identifying and categorizing entities within a text into predefined categories such as the names of persons, organizations, locations, dates, etc.

In the [entities wiki](#), you have seen how to create your own entities, which then are embedded into [intent parameters](#). The NER component of an agent will be in charge of recognize them within the user messages.

Simple NER

This is the NER used by the [Simple Intent Classifier](#). It extracts parameters from the user messages (only when using intents with parameters).

It is a simple algorithm that gets all possible parameter's values and checks if they are present in the messages, thus it will only be able to successfully recognize entities when there is an **exact match** with one of the parameter values.

For instance, if an entity 'sport' has a value 'football', and the user writes 'I like foot ball', there will be no parameter matching since 'foot ball' is not a value in 'sport' ('football' is)

The Simple NER can also recognize [Base entities](#), which don't have a predefined set of values and are more generic.

LLM NER

The [LLM Intent Classifier](#) does also the NER task, extracting the possible parameters in a user message. In comparison with the Simple NER, this NER is able to recognize parameters **without exactly matching an entity value**.

For instance, given an entity 'sport' with a value 'football' and with no synonyms, if the user writes 'I like foot ball', the parameter 'football' will probably be recognized. It also works with synonyms

(that don't necessarily have to be defined in the entities). For the sentence 'I like soccer', the LLM NER would probably recognize 'football'.

In addition, with LLM NER you can define your entities without values and **simply add an entity description** (or use a combination of both). Let's see an example entity:

```
email_entity = agent.new_entity(  
    name='email_entity',  
    description='An entity containing email addresses',  
)
```

LLM

A Large Language Model (LLM) is a type of Artificial Intelligence that is designed to understand and generate human language. These models are trained on vast amounts of text data and use complex algorithms, particularly deep learning techniques, to learn the patterns and structures of language.

There are multiple kinds of LLMs, so choosing one may become a complicated task if we are not familiar with them. Moreover, the NLP field moves at a very fast pace, so LLMs become obsolete quite fast (which is actually good because it means something better has been created out there!).

Some important things to consider when selecting an LLM:

- **Proprietary** (e.g., OpenAI) vs **Open-Source** (HuggingFace is the biggest repository): Most proprietary LLMs are good and reliable, although it is rare to find good free options.
- **Locally deployed vs through API**: You can use an LLM deployed on your premises, but depending on its size it may be cheaper to pay for an API (The HuggingFace Inference API offers some LLMs for free)
- **Latency**: the time the LLM spends to generate the answers.
- **Size**: Smaller LLMs (e.g., 2 billion parameters) tend to be more limited than bigger ones (e.g., 70 billion parameters), although they may be enough if you want to deploy locally and for specific tasks.

How to use

Let's see how to seamlessly integrate an LLM into our agent. You can also check the [LLM agent](#) for a complete example.

We are going to create an LLMOpenAI:

```
from baf.nlp.llm.llm_openai_api import LLMOpenAI  
  
agent = Agent('example_agent')  
  
gpt = LLMOpenAI(agent=agent, name='gpt-4o-mini')
```

This LLM can be used within any agent state (in both the body and the fallback body):

```
def answer_body(session: Session):  
    answer = gpt.predict(session.message) # Predicts the output for the given  
    input (the user message)
```

```
session.reply(answer)
```

There are plenty of possibilities to take advantage of LLMs in an agent. The previous is a very simple use case, but we can do more advanced tasks through prompt engineering.

Adding context information to an LLM

To improve / customize the LLM's behavior, it is also possible to add context information to an LLM. Here, we differentiate between a global context and user-specific context information. The global context will be applied to every LLM prediction for any user. The user specific context is only applied for a specific user and can contain user specific information to personalize the LLM's behavior.

Here an example where we extend the previous LLMOpenAI instance:

```
# adding this global_context will cause the LLM to only answer in english.
gpt = LLMOpenAI(agent=agent, name='gpt-4o-mini', global_context='You only speak english.')
```

Let's now suppose we have access to the user's name while executing the body of the current state:

```
def answer_body(session: Session):
    user_name = session.message
    # For this specific session, the LLM will always be given the "context"
    string as additional information.
    gpt.add_user_context(session=session, context=f"The user is called
{user_name}", context_name="user_name_context")
    answer = gpt.predict(session=session.message) # Predicts the output for the given
input (the user message)
    # It's also possible to remove context elements from the user specific
context.
    gpt.remove_user_context(session=session, context_name="user_name_context")
    session.reply(answer)
```

It is also possible to only add context information for a specific prompt:

```
def answer_body(session: Session):
    user_name = session.message
    answer = gpt.predict(session=session.message, system_message=f'Start your
response using the name of the user which is {user_name}')
    session.reply(answer)
```

Available LLMs

BAF comes with LLM wrappers that provide the necessary methods to use them. All LLM wrappers must implement the [LLM](#) class, which comes with the following methods to be implemented:

- [initialize\(\)](#): Initialize the LLM.
- [predict\(\)](#): Generate the output for a given input.
- [chat\(\)](#): Simulate a conversation. The LLM receives previous messages to be able to continue with a conversation. Necessary to get chat history from the [database](#). Not mandatory to implement.

- [`intent\_classification\(\)`](#): Predict the intent of a given message (it allows the [LLM Intent Classifier](#) to use this LLM). Not mandatory to implement.

These are the currently available LLM wrappers in BAF:

- [LLMOpenAI](#): For [OpenAI](#) LLMs
- [LLMHuggingFace](#): For [HuggingFace](#) LLMs locally deployed
- [LLMHuggingFaceAPI](#): For HuggingFace LLMs, through its [Inference API](#)
- [LLMReplicate](#): For [Replicate](#) LLMs, through its API

Note

Models taken from Huggingface or Replicate might expect a specific prompting or context specification format to improve the results. Be sure to carefully read the guidelines for each model for an optimal experience.

RAG

Retrieval-augmented generation (RAG) is a technique for enhancing the accuracy and reliability of generative AI models with facts fetched from external sources.

The benefits of RAG include:

- **Updated information**: provide LLMs with the latest information (LLMs are trained with data until a certain date)
- **Access to domain-specific information** not seen during training without the need of fine-tuning the LLM.
- **Factual grounding**: reduce hallucinations by providing LLMs with access to a knowledge base

The key components of RAG are:

- **Embeddings model**: A model trained to generate an embedding for a given input. This embedding is a high-dimensional vector that captures the information about the input, such as the semantics. This way, we can determine the similarity between 2 inputs by calculating the distance between their embeddings.
- **Vector Store**: It is a specific kind of database for efficient storage, indexing and querying of vector embeddings, or numerical representations of unstructured data, such as text, images or audio.
- **Retriever**: Given an input, it gets the most similar chunks from the vector store by comparing the vector embeddings.
- **Generator (LLM)**: It receives the original query and the retrieved data, and generates the answer.

BAF allows you to integrate this process into your agent. Our implementation uses [LangChain](#), which is a framework for developing apps with LLMs. It comes with a wide library of resources we can use to customize our RAG's components.

Let's see how to seamlessly integrate a RAG component into our agent. You can also check the [RAG agent](#) for a complete example.

First of all, we create our agent:

```
from baf.core.agent import Agent

agent = Agent('greetings_agent')
agent.load_properties('config.yaml')
websocket_platform = agent.use_websocket_platform(use_ui=True)
```

Now, we have to create the RAG components using the LangChain library.

We will need:

Embeddings

An embeddings model. We can use an OpenAI embeddings as an example:

```
from langchain_community.embeddings import OpenAIEmbeddings

embeddings = OpenAIEmbeddings(openai_api_key='api-key')
```

Note

- Full list of LangChain [Embeddings](#) implementations.

Vector Store

A VectorStore. We can use the Chroma vector store as an example:

```
from langchain_community.vectorstores import Chroma

vector_store: Chroma = Chroma(
    embedding_function=embeddings,
    persist_directory='vector_store' # directory where we store the vector
store, optional
)
```

Note

- Full list of LangChain [VectorStore](#) implementations.

Text Splitter

A TextSplitter to divide the documents into smaller chunks (only necessary if we want to load data into the vector store). LangChain provides different splitters for specific splitting criteria:

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=100)
```

Note

- Full list of LangChain [TextSplitter](#) implementations.

LLM

An LLM, using the BAF [LLM wrappers](#):

```
from baf.nlp.llm.llm_openai_api import LLMOpenAI

gpt = LLMOpenAI(agent=agent, name='gpt-4o-mini')
```

RAG

Now we can create the RAG

```
rag = RAG(
    agent=agent,
    vector_store=vector_store,
    splitter=splitter,
    llm_name='gpt-4o-mini',
    k=4, # Number of chunks to retrieve
    num_previous_messages=0 # Number of previous messages to add to the query
)
```

Note

The [API docs](#) contain full details on the RAG parameters

Import data

If you want to load data into the vector store, our implementation with LangChain's PDF loader:

```
rag.load_pdfs('./pdfs')
```

Or use any of the LangChain's [document loaders](#), for instance:

```
from langchain_community.document_loaders import TextLoader

loader = TextLoader("./index.md")
documents = loader.load()
chunked_documents = splitter.split_documents(documents)
vector_store.add_documents(chunked_documents)
```

Execution

Finally, let's use the RAG within a state (it can be used in both the body and the fallback body):

```
def rag_body(session: Session):
    # Option 1: it uses the last user message by default
    rag_message: RAGMessage = session.run_rag()
    # Option 2: use a custom message as input
    rag_message: RAGMessage = session.run_rag(message='custom message')
    # Option 3: run RAG without the session
    rag_message: RAGMessage = rag.run(message='custom message')

    # Reply the generated answer
    session.reply(rag_message.answer)
```

```
# Or a specific method to reply RAG messages (to display the answer and the
retrieved documents)
websocket_platform.reply_rag(session, rag_message)
```

A [RAGMessage](#) is the return object of the RAG. It contains the generated answer together with the retrieved documents, and additional metadata.

The [WebSocket platform](#) includes a method to reply this kind of messages, and our Streamlit UI can display them within expander containers that show the retrieved documents to the user.

Speech-to-Text

BAF allows you to use your voice to interact with the agents, transforming them into voicebots! To do this, it implements a Speech-to-Text component (also known as automatic speech recognition, STT or S2T). It solves the NLP task of transcribing an audio file. Then, the transcription is treated as a typical user text message.

Available Speech-to-Text models

BAF supports a variety of implementations for speech-to-text:

- [HFSpeech2Text](#): For [HuggingFace](#) STT models. Example model: `openai/whisper-tiny` (very lightweight model)
- [APISpeech2Text](#): For the [SpeechRecognition](#) Python library. Currently only supports Google Speech Recognition.
- [OpenAISpeech2Text](#): For [OpenAI](#) STT models. Example model: `whisper-1`
- [LuxASRSpeech2Text](#): For the [LuxASR](#) API.

Optional parameters for LuxASR [speech2text\(\)](#):

- `mime_type`. Example: `audio/mpeg` or `plain/text`, defaults to `application/octet-stream` for voice inputs.
- `diarization`. Example `Enabled` (default)
- `output_format`. Example `text` (default)

How to use

Let's see how to seamlessly integrate a Speech2Text model into our agent. You can also check the [Speech2Text agent](#) for a complete example.

We are going to implement the HFSpeech2Text class. We start by creating our Agent and define the STT model(s):

```
agent = Agent('example_agent')

# Define STT and TTS Models
stt = HFSpeech2Text(agent=agent, model_name="openai/whisper-tiny")
```

The Agent builds the NLP Engine, which implements the corresponding Speech2Text class in the background (eg. HFSpeech2Text). The Speech2Text component will be automatically called when the user speaks into the microphone through the Streamlit UI and the transcribed message can be used within any agent state and simply accessed through the Session parameter.

```
def stt_body(session: Session):  
    session.reply("User: " + session.event.message)
```

There are plenty of possibilities to take advantage of Speech2Text models in an agent. The previous is a very simple use case, but we can do more advanced tasks by combining the Speech2Text module with an [LLM](#).

Combining Speech2Text with LLM responses

It is possible to simulate a conversation by passing the spoken user message to an LLM after it has been transcribed by the Speech2Text model. The following shows an example where the transcribed message is passed to an LLM:

First, let's define the LLM:

```
gpt = LLMOpenAI(  
    agent=agent,  
    name='gpt-4o-mini',  
    parameters={},  
    num_previous_messages=100,  
)
```

Within any agent state, the transcribed message can be passed to the LLM by accessing the Session parameter:

```
def stt_body(session: Session):  
    session.reply("User: " + session.event.message)  
    answer = gpt.predict(session.event.message)  
    session.reply(answer)
```

Text-to-Speech

BAF allows you to create synthetic speech from your texts! To do this, it implements a Text-to-Speech component (also known as speech synthesis, TTS or T2S). It solves the NLP task of converting written text into audio speech signals.

Available Text-to-Speech models

BAF supports a variety of implementations for text-to-speech:

- [HFText2Speech](#): For [HuggingFace](#) TTS models. Example model: facebook/mms-tts-eng Optional parameters for [text2speech\(\)](#): - return_tensor. Example pt (default)
- [OpenAIText2Speech](#): For [OpenAI](#) TTS models. You can set the optional class parameter voice (default alloy). Example model gpt-4o-mini-tts.

- [PiperText2Speech](#): For the [Piper](#) TTS implementation (Only tested with the HuggingFace Model mbarnig/lb_rhasspy_piper_tts). You need to download the model and run it through a Docker container as Piper currently only works on Linux. Example Model: mbarnig/lb_rhasspy_piper_tts (default).

How to use

Let's see how to seamlessly integrate a Text2Speech model into our agent. You can also check the [Text2Speech agent](#) for a complete example.

We are going to implement the HFText2Speech class. We start by creating our Agent and defining the TTS model(s):

```
agent = Agent('example_agent')

tts = HFText2Speech(agent=agent, model_name="facebook/mms-tts-eng")
```

The Agent builds the NLP Engine, which implements the corresponding Text2Speech class in the background (eg. HFText2Speech). The TTS component is automatically called through the Websocket Platform `reply_speech()` function. When called, it returns the synthesised audio which is then send back to the user as an audio message:

```
def tts_body(session: Session):
    websocket_platform.reply_speech(session, session.event.message)
```

Speech-to-Speech

BAF allows you to simulate a spoken conversation with an agent! To do this, it implements a Speech-to-Text component (also known as automatic speech recognition, STT or S2T) and combines it with a Text-to-Speech component (also known as speech synthesis, TTS or T2S). It solves the NLP task of transcribing an audio file, treating the transcription as a typical user text message and sending it to an LLM, then converting the text answer by the LLM into audio speech signals.

How to implement

Let's see how to seamlessly integrate a Speech2Speech component into our agent. You can also check the [Speech2Speech agent](#) for a complete example.

We are going to implement the HFText2Speech class. We start by creating our Agent and set the STT/TTS models:

```
agent = Agent('example_agent')

# Define STT and TTS Models
stt = OpenAISpeech2Text(agent=agent, model_name="whisper-1", language="en")
stt2 = OpenAISpeech2Text(agent=agent, model_name="gpt-4o-mini-transcribe")
tts = OpenAIText2Speech(agent=agent, model_name="gpt-4o-mini-tts", language="en", voice="coral")
```

```
tts2 = OpenAIText2Speech(agent=agent, model_name="gpt-4o-mini-tts",
language="fr", voice="ash")
stt_lux = LuxASRSpeech2Text(agent=agent, language="lb")
piper = PiperText2Speech(agent, language="lb")
```

The Agent implements the corresponding Speech2Text and Text2Speech classes in the background (eg. OpenAISpeech2Text, OpenAIText2Speech). It is possible to simulate a spoken conversation by passing the spoken user message to an LLM after it has been transcribed by the Speech2Text model. The following shows an example where the transcribed message is passed to an LLM and the answer provided by the LLM is returned back as a spoken message.

First, let's define the LLM:

```
gpt = LLMOpenAI(
    agent=agent,
    name='gpt-4o-mini',
    parameters={},
    num_previous_messages=100,
    global_context='You are a helpful assistant. Always match and answer in the
language the user is speaking to you. '
    'Keep your answers concise and to the point. Do not use any
formatting or bullet points.',
)
```

Within any agent state, the transcribed message can be passed to the LLM by accessing the Session parameter (use the chat() function to store the chat history. The response by the LLM is then passed to the TTS component and send back to the user as an audio message, all happening within the Websocket Platform reply_speech() function:

```
def stt_message_body(session: Session):
    # only transcribe message if the user spoke
    if isinstance(session.event, ReceiveJSONEvent) or isinstance(session.event,
ReceiveMessageEvent):
        session.reply("User: " + session.event.message)
        answer = gpt.chat(session)
        websocket_platform.reply_speech(session, answer)
        session.reply(answer)
```

Adding Files to a Speech2Speech Agent

It is possible to have the user upload files through the Streamlit UI, containing either a spoken or a written message. We can check for the mime type to differentiate between a Speech2Text or a Text2Speech task:

```
# Execute when a file is received
def stt_file_body(session: Session):
    # get user language
    lang = session.get("user_language", "en")
    # access STT system based on language mapping
    s2t = session._agent._nlp_engine._language_to_speech2text_module[lang]
    event: ReceiveFileEvent = session.event
    file: File = event.file

    # Determine MIME type
    ext = file.name.lower()
    # do only for text files
    if ext.endswith(".txt"):
        mime_type = "text/plain"
```

```

elif ext.endswith(".wav"):
    mime_type = "audio/wav"
elif ext.endswith(".mp3"):
    mime_type = "audio/mpeg"
elif ext.endswith(".m4a"):
    mime_type = "audio/mp4"
else:
    mime_type = "application/octet-stream"

# only when audio files are uploaded
if not mime_type == "text/plain":

    # convert file to byte representation
    base64_content = file._base64
    # Decode the base64 string into bytes
    file_bytes = base64.b64decode(base64_content)
    # add to logger
    logger.info(f"Successfully decoded {len(file_bytes)} bytes.")
    # for the LuxASR model, we need to indicate the MIME Type
    if lang == "lb":
        # call LuxASR Speech2Text and get transcription
        text = s2t.speech2text(file_bytes, mime_type)
    else:
        text = s2t.speech2text(file_bytes)
    session.reply("User: " + text)
    answer = gpt.predict(text)
    #session.reply(answer)
    file_text = answer
else:
    # convert file to byte representation
    base64_content = file._base64
    # Decode the base64 string into text
    file_text = base64.b64decode(base64_content).decode('utf-8')

# call Speech2Text and get transcription
session.reply(file_text)
websocket_platform.reply_speech(session, file_text)

```